Antigravity Prompt Sheet

Algerian Travel Agency Platform - what to paste for each phase

How to use this sheet

- Work in order: Step 0, then Phase 0, 1A, 1B, 1C, 1D, 1E, 2, and the final hardening pass.
- Start a NEW conversation for every step, in Planning mode. The rules and docs/PROGRESS.md carry the context.
- Each step has two options. Option A is a one-line message if you installed the workflow files. Option B is the full prompt if you did NOT install the workflows.
- The five rule files (.agents/rules/) apply to both options. Never paste the master prompt: it already lives in the rules.
- Before moving on, do the 'Check before moving on' list yourself and read the deviations in docs/PROGRESS.md.

Files that must be in your project first

```
your-repo/
  docs/
    spec.pdf
    PROGRESS.md
  .agents/
    rules/ 00-project.md 01-security.md 02-business-rules.md
                 03-performance.md 04-design.md
    workflows/ (optional) 00-implementation-plan.md, phase-0 ... phase-9
```

Machine prerequisites

- Node.js LTS, pnpm and Docker Desktop installed (Docker runs PostgreSQL).
- A Git repository initialized, so you can roll back a phase that goes wrong.
- In Antigravity: Customizations > Rules shows 00 to 03 as Always On and 04 as Glob.

Common prefix used in Option B prompts

Option B prompts start by telling the agent to read the project files and end by asking it to verify and update PROGRESS.md. This replaces what the workflow files do automatically.

Step 0 - Implementation Plan (before any code)

Option A - with workflows installed: paste this

```
/00-implementation-plan Use @/docs/spec.pdf @/docs/PROGRESS.md
```

Optional details to add under the command: agency name, languages at launch (Arabic + French), services sold (outbound trips, Umrah, visa assistance, hotels), brand colors or logo. If unknown, write 'use placeholders'.

Option B - without workflows: paste this full prompt

```
Read @/docs/spec.pdf completely and @/docs/PROGRESS.md.
Produce an Implementation Plan artifact containing: architecture, monorepo folder structure, the
complete relational data model (all entities from spec section 10.2 as SQL tables with columns,
foreign keys, constraints and indexes, plus an ER diagram), the RBAC permission matrix (roles x
modules x actions), the API route list, the design system outline, the phase breakdown with
acceptance criteria, and a list of risks and open questions.
Explain specifically how money (NUMERIC with a currency column), reservation price snapshots, atomic
reference generation and capacity checks (transactions, row locks, constraints) will be modeled in
PostgreSQL.
Do NOT write application code. Wait for my explicit approval.
After my approval, save the plan as /docs/PLAN.md and mark "Plan approved" in /docs/PROGRESS.md.
```

Check before moving on

- Read the data model and the permission matrix line by line. Ask for changes until correct.
- Check that money is stored as NUMERIC with a currency column (never floats), and that reservations store a price snapshot.
- Reply 'Approved' only when satisfied. Confirm docs/PLAN.md exists afterwards.

Phase 0 - Foundation

Option A - with workflows installed: paste this

```
/phase-0-foundation Use @/docs/spec.pdf @/docs/PLAN.md @/docs/PROGRESS.md
```

Optional lines to add: 'Agency name: [name]. Languages: Arabic + French. Services: [list]. Use placeholder branding for now.'

Option B - without workflows: paste this full prompt

```
Read @/docs/spec.pdf (sections 6, 9, 10), @/docs/PLAN.md and @/docs/PROGRESS.md. Confirm the plan is
approved; if not, stop.
Implement Phase 0: pnpm monorepo (apps/web, apps/admin, apps/api, packages/shared), strict
TypeScript, ESLint, Prettier, Husky pre-commit, .gitignore.
Docker Compose with PostgreSQL 16 (persistent volume), the API and dev tooling. Set up Drizzle ORM
with drizzle-kit migrations, the pg connection pool, a non-superuser application database role, and
migration and seed scripts. Provide .env.example and Zod env validation that stops startup on
misconfiguration.
CI workflow: lint, typecheck, test, pnpm audit. Tests use Vitest, Supertest and Testcontainers (real
PostgreSQL).
Express skeleton: helmet with strict CSP, CORS allow-list, rate limiting, hpp, compression,
centralized error handling, pino logging, health endpoint, graceful shutdown.
Tailwind design tokens and the base design system (buttons, inputs, cards, dialogs, toasts, tables,
badges) in Arabic and French with RTL/LTR switching and a language switcher. Use Framer Motion via
LazyMotion.
Add the seed script structure and a living style-guide page so I can review the design system.
Run lint, typecheck, tests and build. Start the stack and verify in the browser with screenshots.
Check the result against .agents/rules/01-security.md and 03-performance.md and list any gap.
Update /docs/PROGRESS.md and summarize.
```

Check before moving on

- docker compose up starts without errors and the health endpoint answers.
- The style-guide page opens and the language switcher flips correctly between RTL and LTR.
- Lint, typecheck, tests and build all pass. PROGRESS.md shows Phase 0 done.

Phase 1A - Auth, RBAC, audit log, staff management

Option A - with workflows installed: paste this

```
/phase-1a-auth Use @/docs/spec.pdf @/docs/PLAN.md @/docs/PROGRESS.md
```

Option B - without workflows: paste this full prompt

```
Read @/docs/spec.pdf (sections 4.2, 9, 12.1), @/docs/PLAN.md and @/docs/PROGRESS.md. Confirm Phase 0
is done; if not, stop.
Implement users, roles and the permission matrix from /packages/shared; argon2id; cookie sessions
with rotating refresh tokens and reuse detection; CSRF protection; login throttling and lockout;
password reset; account deactivation; session revocation.
Implement the append-only AuditLog service (UPDATE/DELETE blocked by a database trigger) and use it
for logins, failed logins and permission changes.
Backend RBAC middleware on every protected route, plus object-level authorization helpers.
Admin app: login page, staff management UI (create, edit role, deactivate), protected routing,
session handling.
Write tests proving: unauthenticated requests are rejected; each role reaches only its permitted
routes; a deactivated user is cut off immediately; brute-force login is throttled; refresh token
reuse revokes the session.
Run lint, typecheck, tests and build, verify in the browser, check against
.agents/rules/01-security.md, list gaps, update /docs/PROGRESS.md and summarize.
```

Check before moving on

- Log in as each role and confirm each only sees what the permission matrix allows.
- Deactivate a user and confirm their session stops working immediately.
- Confirm the security tests pass and the failed-login attempts appear in the audit log.

Phase 1B - Offers, customers (CRM), enquiries

Option A - with workflows installed: paste this

```
/phase-1b-offers-customers-enquiries Use @/docs/spec.pdf @/docs/PLAN.md @/docs/PROGRESS.md
```

Option B - without workflows: paste this full prompt

```
Read @/docs/spec.pdf (sections 2.2, 2.3, 3.2, 3.3), @/docs/PLAN.md and @/docs/PROGRESS.md. Confirm
Phase 1A is done; if not, stop.
Offer management: create, edit, publish, unpublish, archive; destination, trip type, dates,
duration, DZD price, pricing basis, capacity, accommodation, transport, description, day-by-day
itinerary, included/excluded services, photo gallery through the sharp image pipeline, downloadable
brochure, availability labels (Available, Limited places, Request availability, Fully booked).
Public catalogue API: published offers only, filters (destination, trip type, date range, budget),
pagination, caching headers.
Customer CRM: create, view, edit, archive, search; preferred language; internal notes; restricted
sensitive traveller fields; duplicate detection by phone/email with staff review; full history view.
Enquiry management: public submission endpoint (honeypot, rate limit, idempotency key, consent
record) and manual creation (phone, walk-in, WhatsApp); assignment, follow-up date, statuses,
activity timeline, internal notes.
Admin UI for all of the above with filterable, paginated tables. Add all required PostgreSQL indexes
in migrations (including pg_trgm for search).
Tests: validation errors, duplicate submission creates no duplicates, content editor cannot read
customers, IDOR attempts fail.
Run lint, typecheck, tests and build, verify in the browser, check against
.agents/rules/01-security.md and 03-performance.md, list gaps, update /docs/PROGRESS.md and
summarize.
```

Check before moving on

- Create and publish an offer with photos. Unpublish it and confirm it disappears from the public API.
- Submit an enquiry twice quickly and confirm only one is created.
- Create two customers with the same phone number and confirm the duplicate warning appears.

Phase 1C - Reservations, payments, receipts

Option A - with workflows installed: paste this

```
/phase-1c-reservations-payments Use @/docs/spec.pdf @/docs/PLAN.md @/docs/PROGRESS.md
```

Option B - without workflows: paste this full prompt

```
Read @/docs/spec.pdf (sections 3.4, 3.6, 8.1, 8.2), @/docs/PLAN.md and @/docs/PROGRESS.md. Confirm
Phase 1B is done; if not, stop.
Reservations: create from an enquiry or directly; link customer, offer, travellers and responsible
employee; price SNAPSHOT; authorized discount; special requirements; unique reference via atomic
counter; status tracking with full history; printable confirmation.
Payments: agreed total, deposits, instalments, remaining balance computed server-side; record date,
amount, method, reference and the entering employee; configurable methods; due-date flags; refunds
and adjustments as explicit records; no hard deletes.
Receipt and confirmation PDFs in Arabic and French with correct RTL layout and an embedded Arabic
font.
Use PostgreSQL transactions (with SELECT ... FOR UPDATE where needed) for every operation that
touches money, reservation numbering or capacity. Add CHECK constraints and triggers that block
UPDATE/DELETE on payments, refunds and audit rows.
Admin UI: reservation list and detail with status timeline, payment panel, receipt download.
Tests: balance is correct after multiple instalments and refunds; changing an offer price does not
alter existing reservations; repeated submission does not duplicate reservations; finance vs agent
permissions; references are unique under concurrency.
Run lint, typecheck, tests and build, verify in the browser, check against
.agents/rules/01-security.md and 02-business-rules.md, list gaps, update /docs/PROGRESS.md and
summarize.
```

Check before moving on

- Convert an enquiry into a reservation. Record two payments and one refund; verify the balance by hand.
- Change the offer price and confirm the existing reservation keeps its agreed price.
- Open the Arabic receipt PDF and check the right-to-left layout and the DA amounts.
- Try to edit or delete a payment and confirm it creates a reversal/audit record instead.

Phase 1D - Public website (UI/UX, SEO, performance)

Option A - with workflows installed: paste this

```
/phase-1d-public-website Use @/docs/spec.pdf @/docs/PLAN.md @/docs/PROGRESS.md
```

Optional line to add: 'Design notes: [brand colors, logo, websites you like or dislike, or none]'. This is where you steer the look.

Option B - without workflows: paste this full prompt

```
Read @/docs/spec.pdf (sections 2, 6, 12), @/docs/PLAN.md and @/docs/PROGRESS.md. Re-read
.agents/rules/04-design.md and 03-performance.md. Confirm Phase 1C is done; if not, stop.
First propose the visual concept (palette, type pairing, homepage section layout, motion plan,
Algeria map approach) as a short artifact with a mockup screenshot. Wait for my approval before
building all pages.
Then build: Home, About, Offers (filters), Offer details (gallery, itinerary, inclusions/exclusions,
enquiry form), Services, Destinations, Contact (map, hours, WhatsApp click-to-chat with prefilled
message), FAQ, Legal pages.
Enquiry form: validation, consent checkbox, honeypot, idempotency key, local draft recovery, a clear
message that a request is not a confirmed reservation, clear confirmation after submit.
Full Arabic (RTL) and French (LTR) with translation files, localized dates and DZD formatting.
SEO: prerendering, hreflang, structured data (TravelAgency, TouristTrip), sitemap, robots, Open
Graph.
Sticky mobile call/WhatsApp/request bar; accessible forms and focus states.
Measure Lighthouse (mobile) and bundle size, fix anything below target and report the real numbers.
Verify in the browser at mobile and desktop widths in both languages with screenshots. Update
/docs/PROGRESS.md and summarize.
```

Check before moving on

- Judge the concept mockup first: is it distinctive and not like other agency sites? Request changes before it builds everything.
- Lighthouse mobile scores are 90+ for Performance, SEO, Accessibility and Best Practices.
- Test the whole site in Arabic and French on a phone-sized window, and submit the enquiry form.

Phase 1E - Dashboard, private documents, reports

Option A - with workflows installed: paste this

```
/phase-1e-dashboard-documents-reports Use @/docs/spec.pdf @/docs/PLAN.md @/docs/PROGRESS.md
```

Option B - without workflows: paste this full prompt

```
Read @/docs/spec.pdf (sections 3.1, 4.1, 4.3, 12.1), @/docs/PLAN.md and @/docs/PROGRESS.md. Confirm
Phase 1D is done; if not, stop.
Dashboard: enquiries awaiting response, active reservations, upcoming departures, tasks needing
attention, payments received vs outstanding for a period, shortcuts (add customer, create offer,
register reservation, record payment), filters by date, employee, trip and status.
Private document uploads attached to a customer or reservation: type, upload date, expiry date,
status; private storage; authenticated streaming or short-lived signed URLs; magic-byte MIME checks,
size limits, randomized names, malware-scan hook; role-based access; log every sensitive access;
download, replace and delete per retention policy.
Reports: reservations by period/destination/offer/status; revenue collected vs amounts due (cash
received clearly separate from booked sales); enquiry conversion; CSV and PDF export; date filters;
financial reports restricted by role.
Tests: an employee cannot open a document or financial record outside their role; document URLs are
never public; exports respect permissions.
Run lint, typecheck, tests and build, verify in the browser, check against
.agents/rules/01-security.md, list gaps, update /docs/PROGRESS.md and summarize. Mark the MVP
complete only if every item in spec section 12.1 passes.
```

Check before moving on

- Upload a test passport scan, then try to open its URL while logged out. It must fail.
- Log in as a content editor and a finance employee and confirm each is limited as the permission matrix says.
- Export a CSV report and confirm cash received is shown separately from booked sales.

Phase 2 - Departures, customer portal, notifications, tasks

Option A - with workflows installed: paste this

```
/phase-2-operations Use @/docs/spec.pdf @/docs/PLAN.md @/docs/PROGRESS.md
```

Option B - without workflows: paste this full prompt

```
Read @/docs/spec.pdf (sections 3.5, 4.4, 5, 7), @/docs/PLAN.md and @/docs/PROGRESS.md. Confirm the
MVP is complete; if not, stop.
Departures and capacity: multiple departures per offer, booked/available places, optional waiting
list, capacity check in a transaction before confirming, manager override with audit, passenger
lists.
Customer portal: secure invitation/booking link, view own reservation, status and balance, protected
document upload, download confirmation/itinerary/receipts, submit a question or correction request.
It must expose only that customer's data.
Notifications: in-dashboard alerts (new enquiries, upcoming departures, missing documents, due
payments); email via a queue with retries and failure-safe handling; WhatsApp click-to-chat
templates; staff can record that a customer was contacted.
Employee assignments and task reminders; extended audit log views.
Tests: a customer cannot access another customer's reservation by changing an identifier; capacity
cannot be exceeded without a manager override; notification failures do not break the main
operation.
Run lint, typecheck, tests and build, verify in the browser, check against
.agents/rules/01-security.md and 02-business-rules.md, list gaps, update /docs/PROGRESS.md and
summarize.
```

Check before moving on

- As a portal customer, change the reservation ID in the URL and confirm access is denied.
- Try to confirm one place beyond capacity as an agent (must fail) and as a manager (override is audited).
- Switch off the email service and confirm that saving a reservation still works.

Final hardening - security, performance, acceptance

Option A - with workflows installed: paste this

```
/phase-9-final-hardening Use @/docs/spec.pdf @/docs/PLAN.md @/docs/PROGRESS.md
```

Option B - without workflows: paste this full prompt

```
Read @/docs/spec.pdf (sections 9, 12.1, 13), @/docs/PLAN.md and @/docs/PROGRESS.md.
Security review of the whole codebase against .agents/rules/01-security.md: check every route for
missing authorization and IDOR, verify CSP, cookie flags, CORS, rate limits, file-serving rules,
audit coverage and secrets handling. Run pnpm audit and fix or document findings.
Verify that every "Minimum test scenario" in spec section 9 exists as an automated test; write the
missing ones and run them all.
Performance review: Lighthouse mobile for key public pages, bundle analysis, API response times on
seeded data with realistic volume, index usage with EXPLAIN (ANALYZE).
Demonstrate backup and restore: write a documented script using pg_dump and pg_restore, back up the
database, restore into a test environment, verify data integrity.
Run the acceptance checklist from spec section 12.1 item by item with evidence (screenshots, test
names).
Write /docs/HANDOVER.md: setup, deployment, environment variables, backup/restore, user roles guide,
support process.
Output a final pass/fail checklist report with evidence, then update /docs/PROGRESS.md.
```

Check before moving on

- Read the final report and spot-check three claims yourself (one security, one performance, one business rule).
- Confirm the backup was restored in a test environment and the data matches.
- Have the agency's accountant validate the receipt format before any real use.

If the rules do not seem to load

- Start a new conversation and ask: 'Which rules are you following for this project?' The agent should mention NUMERIC money (never floats), PostgreSQL transactions, RBAC on every route, the performance targets and the design direction.
- If it cannot, open Customizations > Rules and set each rule to Always On (04-design.md to Glob).
- As a fallback, paste the full contents of antigravity-master-prompt.md at the start of the conversation, or save it as a global rule in ~/.gemini/GEMINI.md.
- Workflow files are limited to 12,000 characters each. Keep them under that if you edit them.

Reminder

The finished software must not be presented as legally or tax compliant. Have the agency and its accountant or legal adviser validate receipt formats, privacy notices and contract wording, as the specification itself requires.